

Informe Técnico Final — Asistente Conversacional Alimentos Cárnicos S.A.S.

Proyecto: Carnicos Knowledge Base

Entrega: Informe Unificado — Módulos 1, 2 y 3

Fecha: Junio 2026

Canal de despliegue: WhatsApp vía Twilio · N8N · ngrok · FastAPI

Integrantes:

- Carlos Alex Macias Perdomo — código: 22500208
 - Carlos Hidalgo Escobar — código: 22502395
 - Lorena Portilla — código: 22500248
 - Luis Carlos Correa — código: 22501541
-

1. Problema y Solución

1.1 Necesidad identificada

Alimentos Cárnicos S.A.S. (Grupo Nutresa) cuenta con información pública distribuida en múltiples fuentes digitales: sitio web corporativo, secciones institucionales, documentos PDF de políticas, informes de sostenibilidad y formularios de atención. Para un usuario externo por ejemplo consumidor, cliente institucional o interesado comercial. Poder consultar esta información exige navegar varias páginas o documentos, lo que dificulta obtener respuestas rápidas, consistentes y en el canal de comunicación que el usuario ya utiliza a diario: WhatsApp.

El problema central es la ausencia de un canal automatizado de primer contacto que responda con precisión preguntas frecuentes sobre la empresa, sus productos, marcas, sedes, canales de atención y políticas, sin reemplazar procesos transaccionales reales ni exigir al usuario cambiar de plataforma.

1.2 Solución planteada

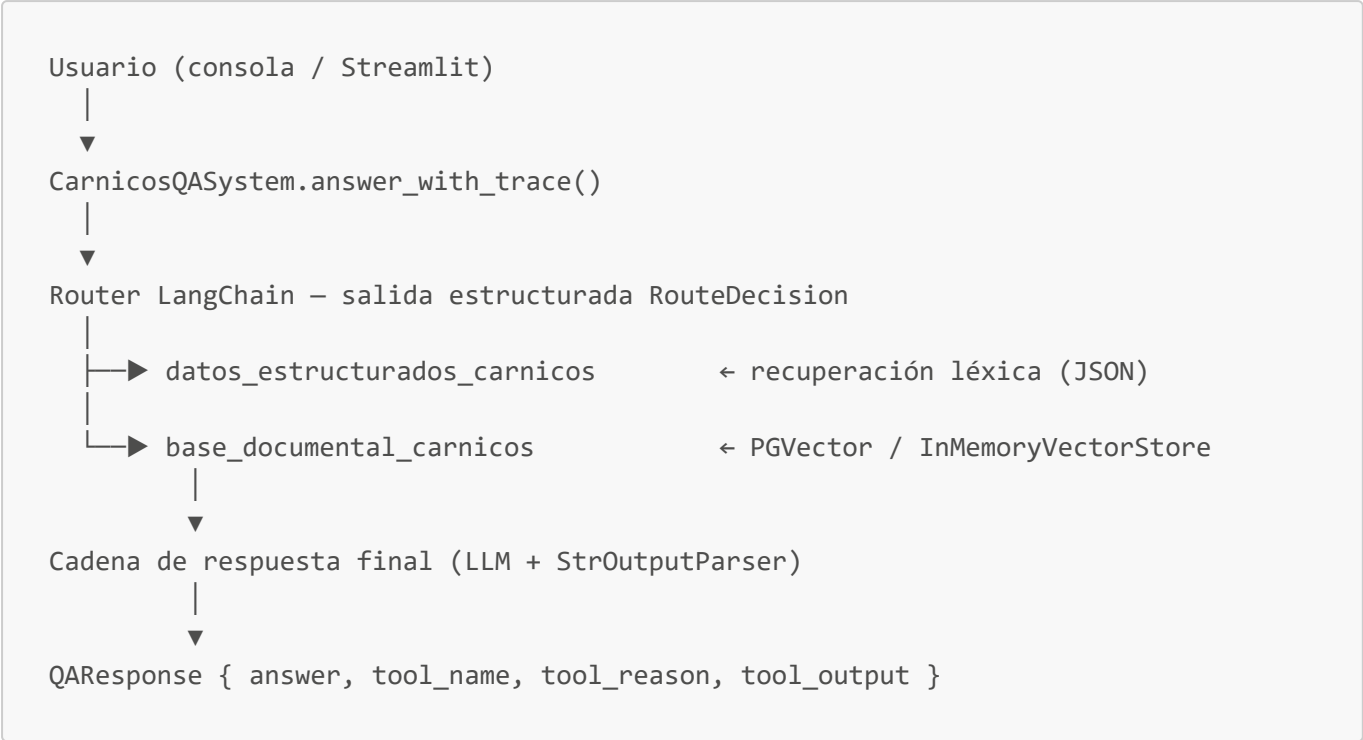
La solución construida a lo largo de tres módulos es un asistente conversacional basado en LLM que:

1. Recopila y procesa la información pública de la empresa (web scraping + PDFs).
 2. La convierte en una base de conocimiento indexada con embeddings vectoriales (RAG).
 3. Expone el agente a través de una API REST (FastAPI) publicada en internet con ngrok.
 4. Orquesta el flujo de mensajes de WhatsApp mediante N8N, usando Twilio como gateway SMS/WhatsApp.
 5. Da la posibilidad mas adelante de incorporar un mecanismo de supervisión humana (HITL) para consultas sensibles o atención personalizada.
-

2. Evolución Arquitectónica: Módulo 2 → Módulo 3

2.1 Arquitectura del Módulo 2

El Módulo 2 introdujo el patrón de agente con herramientas, superando la estrategia de contexto completo en prompt del Módulo 1. El flujo era:



Componentes clave del Módulo 2:

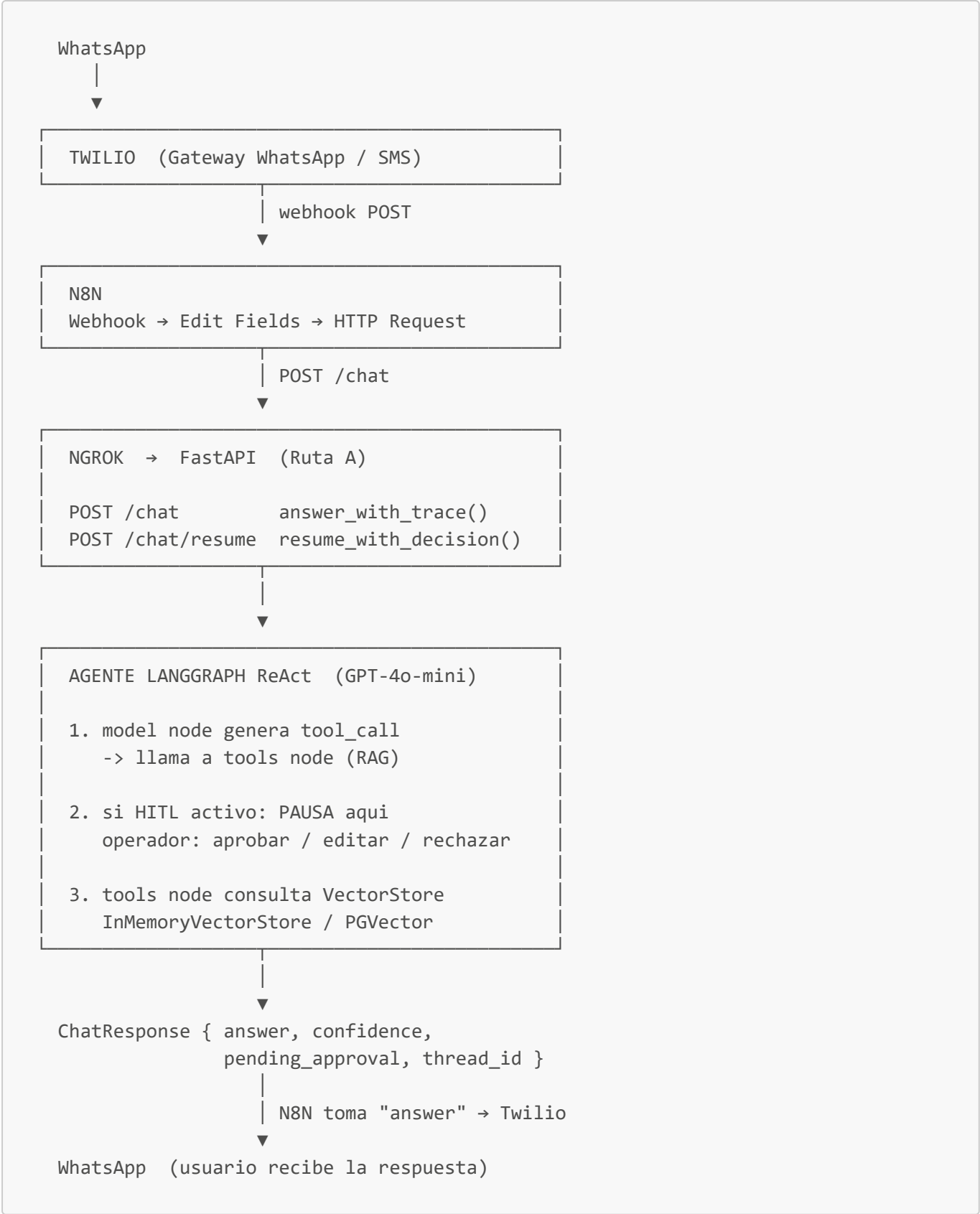
Componente	Implementación
Router	ChatPromptTemplate + with_structured_output(RouteDecision)
Herramienta léxica	structured_data_tool.py — JSON + puntuación de tokens
Herramienta documental	vector_retriever_tool.py — PGVector / InMemory
Memoria de sesión	InMemoryChatMessageHistory / st.session_state
Trazabilidad	LangSmith (opcional, por variable de entorno)
Interfaz	Streamlit

Limitaciones del Módulo 2:

- Sin canal externo: el agente solo era accesible desde la interfaz Streamlit o consola.
- Router explícito con dos herramientas fijas (carga de mantenimiento al añadir herramientas).
- Sin supervisión humana antes de ejecutar consultas sensibles.
- Memoria solo en sesión activa, sin persistencia entre reinicios.

2.2 Arquitectura del Módulo 3 (estado actual)

El Módulo 3 unifica el agente en un único grafo LangGraph ReAct, elimina el router explícito, establece las bases para incorporar HITL, expone una API REST y conecta con WhatsApp a través de N8N + Twilio.



Cambios estructurales respecto al Módulo 2:

Aspecto	Módulo 2	Módulo 3
Agente	Router explícito + cadena de respuesta	<code>create_agent</code> LangGraph ReAct unificado

Aspecto	Módulo 2	Módulo 3
Herramientas	2 fijas (léxica + documental)	1 herramienta RAG dinámica (prompt por retriever)
HITL	No implementado	<code>HumanInTheLoopMiddleware</code> con 3 decisiones
Memoria	InMemory por sesión	<code>InMemorySaver</code> / <code>PostgresSaver</code> con <code>thread_id</code>
Canal	Solo Streamlit / consola	WhatsApp vía Twilio + N8N + ngrok
API	No	FastAPI (<code>/chat</code> , <code>/chat/resume</code> , <code>/health</code>)
<code>thread_id</code>	No (sesión temporal)	Número de teléfono del usuario

3. Ruta A: FastAPI + Function Calling estricto + N8N

3.1 Justificación de la elección

El proyecto sigue la Ruta A: servidor propio con FastAPI expuesto a internet mediante ngrok, orquestación del flujo de mensajes con N8N y gateway de WhatsApp con Twilio. Esta decisión se toma sobre tres ejes:

a) Control total del agente

Un servidor propio en FastAPI permite controlar el ciclo de vida del agente LangGraph (inicialización, checkpointing, HITL), algo que no es posible en plataformas de terceros donde el código del agente se ejecuta en entornos administrados con restricciones de estado.

b) N8N vs. servidor propio para la orquestación del flujo

N8N actúa como capa de enrutamiento de mensajes entre Twilio y FastAPI. Esta separación tiene ventajas concretas:

Criterio	N8N	Servidor propio (Flask/FastAPI como gateway)
Lógica de webhook	Visual, sin código	Código adicional de routing
Reintentos y errores	Nativo	Manual
Transformaciones ligeras	Nodos <code>Edit Fields</code>	Python / middleware
Acoplamiento	Bajo (intercambiable)	Alto
Coste de cambio de canal	Cambiar el nodo Twilio	Reescribir el gateway

N8N desacopla el canal (Twilio/WhatsApp) del agente (FastAPI/LangGraph). Si en el futuro se quiere añadir Telegram o un formulario web, solo se agrega un nodo en N8N sin tocar el agente.

c) Ngrok para el desarrollo y presentación

Ngrok publica el servidor FastAPI local como URL HTTPS accesible desde internet. Twilio requiere un webhook HTTPS para enviar los mensajes entrantes. En producción este rol lo ocuparía un servidor en la nube (Railway,

GCP, AWS); en desarrollo y demostración, ngrok es suficiente y elimina la necesidad de desplegar infraestructura.

3.2 Function Calling estricto (LangChain + LangGraph)

El Módulo 3 usa `create_agent` de LangChain con `response_format=AgentResponseSchema`. Esto fuerza al modelo a devolver siempre una respuesta estructurada con tres campos:

```
class AgentResponseSchema(BaseModel):
    answer: str          # respuesta en español para el usuario
    tool_was_called: bool # si se invocó el RAG
    confidence: str      # "high" | "medium" | "low"
```

El agente usa `function calling` nativo de OpenAI: el LLM decide si invocar la herramienta RAG o responder directamente. No existe un router separado como en el Módulo 2; LangGraph maneja el ciclo `model → tools → model` hasta que el modelo genera la respuesta final sin más tool calls pendientes.

4. Diagrama de Flujo Completo



```

1. model node decide si invocar RAG
2. si HITL activo: PAUSA para aprobacion
3. tools node consulta VectorStore
Devuelve: { answer, confidence }

```

JSON { answer, confidence }

```

N8N - IF confidence == "low"?

TRUE (no supo responder)
-> Mensaje al asesor (numero fijo)
    "Consulta requiere asesor: ..."
-> Mensaje al cliente (phone_number)
    "Sera atendido por un asesor en breve"

FALSE (respondio con confianza)
-> Respuesta del agente al cliente
    (answer del JSON)

```

```

TWILIO -> USUARIO
El usuario recibe la respuesta en WhatsApp

```

5. Implementación Detallada

5.1 Capa de datos — Recopilación y procesamiento (Módulo 1)

La base de conocimiento se construyó a partir de información pública de Alimentos Cárnicos S.A.S.:

Fuentes procesadas:

- 43 archivos Markdown extraídos del sitio web ([make scrape](#))
- 6 documentos PDF convertidos a Markdown ([make pdf](#) / [make pdf-fast](#))
- Archivo consolidado: [data/processed/base_conocimiento_chunks.md](#) (554.650 bytes)
- 329 chunks semánticos generados con [RecursiveCharacterTextSplitter](#)

Pipeline de procesamiento:

```

sitemap XML
|
▼ scraper.py (requests + BeautifulSoup + trafilatura)
|
▼ data/processed/dataset_carnicos/*.md
|
▼ chunking.py (limpieza + división por encabezados + control de longitud)
|

```

```

▼ base_conocimiento_chunks.md
|
▼ rag_index_builder.py → OpenAIEmbeddings → PGVector / InMemoryVectorStore

```

5.2 Capa de agente — LangGraph ReAct (Módulo 3)

El agente se construye en `qa_system.py` con `create_agent`:

```

create_agent(
    model=llm,                                # GPT-4o-mini
    tools=[rag_tool],                          # herramienta RAG documental
    middleware=[rag_prompt_middleware,         # inyecta contexto por retriever
                hitl_middleware],             # (si HITL activo)
    checkpointer=checkpointer,                 # InMemorySaver / PostgresSaver
    name="carnicos_qa_agent",
    response_format=AgentResponseSchema,
)

```

El `thread_id` es el número de teléfono del usuario, lo que permite al agente mantener memoria conversacional independiente por usuario sin almacenamiento adicional.

5.3 HITL — Control humano en consultas sensibles

`HumanInTheLoopMiddleware` intercepta el flujo después de que el LLM genera el `tool_call` y antes de que el vector store ejecute la búsqueda. El operador puede:

Decisión	Efecto
Aprobar	La query llega al vector store sin modificación
Editar	El operador reformula la query antes de que llegue al RAG
Rechazar	El usuario recibe una respuesta de rechazo; el RAG no se ejecuta

Las consultas se clasifican automáticamente como CRÍTICA o RUTINARIA según patrones regex (`precios`, `contratos`, `NIT`, `nómina`, `proveedores`, etc.). Esta clasificación es informativa para el operador.

La persistencia del estado durante la espera de la decisión usa `PostgresSaver` en producción e `InMemorySaver` en desarrollo.

5.4 FastAPI — Contrato de la API

```

POST /chat
Body: { "message": str, "phone_number": str }
Response: ChatResponse

POST /chat/resume
Body: { "thread_id": str, "decision": "approve"|"edit"|"reject",
        "message": str, "edited_query": str|null }

```

Response: ChatResponse

GET /health

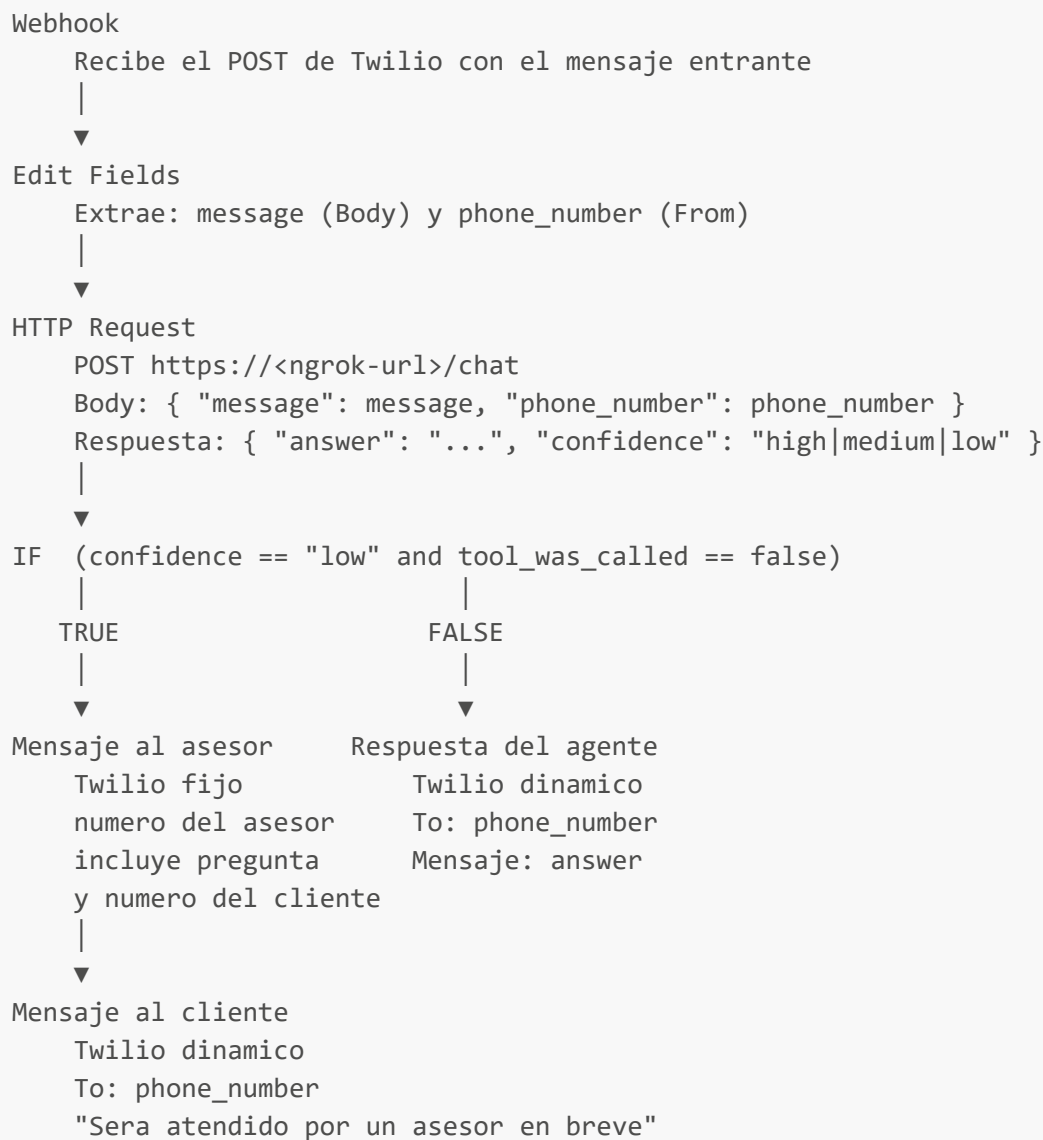
Response: { "status": "ok"|"degraded", "agent_ready": bool }

`ChatResponse` incluye el campo `pending_approval: bool` que N8N puede evaluar para decidir si reenviar la respuesta inmediatamente al usuario o esperar la decisión del operador.

La función `_whatsapp_format()` convierte Markdown estándar al formato de WhatsApp: **`**negrita**`** → **`*negrita*`**, elimina headers `##`.

5.5 Flujo N8N

El flujo N8N tiene seis nodos organizados en una rama principal y dos ramas de salida:



La bifurcación por `confidence` es el mecanismo de escalada: cuando el agente no encuentra evidencia suficiente para responder con seguridad (`confidence = "low"`), N8N redirige automáticamente la consulta a un asesor humano y notifica al cliente. Cuando el agente responde con confianza, el flujo continúa directamente al usuario sin intervención humana.

6. Comparativa de Módulos

Característica	Módulo 1	Módulo 2	Módulo 3
Estrategia de recuperación	Contexto completo en prompt	Router → herramienta léxica o RAG	Agente ReAct con RAG nativo
Embeddings	No	OpenAI <code>text-embedding-3-small</code>	OpenAI <code>text-embedding-3-small</code>
Memoria conversacional	No	<code>InMemoryChatMessageHistory</code>	<code>InMemorySaver</code> / <code>PostgresSaver</code>
<code>thread_id</code> por usuario	No	No	Sí (número de teléfono)
HITL	No	No	Sí (<code>HumanInTheLoopMiddleware</code>)
API REST	No	No	FastAPI (<code>/chat</code> , <code>/chat/resume</code>)
Canal externo	No	No	WhatsApp vía Twilio + N8N
Exposición pública	No	No	ngrok → HTTPS
Respuesta estructurada	No	<code>QAResponse</code> (dataclass)	<code>AgentResponseSchema</code> (Pydantic)
Formato WhatsApp	No	No	<code>_whatsapp_format()</code>
Pruebas unitarias	Chunking (básico)	30 casos, 8 módulos	Contrato API + agente + HITL

7. Conclusiones

7.1 Logros técnicos por módulo

Módulo 1 construyó la base documental: scraping web sistemático, extracción de PDFs en dos modos, pipeline de chunking semántico y un sistema Q&A básico con contexto consolidado en prompt. El resultado fue un corpus de 329 chunks listos para indexación vectorial.

Módulo 2 introdujo el patrón de agente con herramientas y recuperación auditable. El router con salida estructurada garantizó que la elección de herramienta quedara registrada en cada turno, mejorando la trazabilidad y la explicabilidad.

Módulo 3 completó el ciclo hacia producción: agente LangGraph ReAct unificado, API REST documentada, HITL para consultas sensibles, memoria persistente por usuario identificado por número de teléfono, y canal WhatsApp funcional mediante Twilio + N8N + ngrok. Como mejora adicional se implementó escalada

automática a asesor humano: cuando el agente devuelve `confidence = "low"`, N8N notifica al asesor con el contexto de la consulta y confirma al cliente que será atendido, sin ningún cambio en el código del agente.

7.2 Justificación de la arquitectura de despliegue

La separación en tres capas —canal (Twilio), orquestador de mensajes (N8N) y agente (FastAPI + LangGraph) — permite evolucionar cada capa de forma independiente. Cambiar de WhatsApp a Telegram implica solo reemplazar el nodo Twilio en N8N. Cambiar el modelo del agente de GPT-4o-mini a otro LLM implica solo modificar una variable de entorno sin alterar el flujo N8N.

7.3 Limitaciones y próximos pasos

- **ngrok en producción:** reemplazar por despliegue en la nube (Railway, Cloud Run, ECS) con dominio propio y certificado TLS permanente.
- **HITL multi-operador:** la arquitectura actual soporta un único operador por sesión. Producción requeriría un sistema de colas por `thread_id`.
- **Auditoría de decisiones HITL:** las decisiones del operador no se persisten. Una tabla PostgreSQL de auditoría es el siguiente paso para cumplimiento normativo.
- **Escalabilidad del vector store:** `InMemoryVectorStore` es solo para desarrollo. Con `DATABASE_URL` configurada el sistema usa PGVector, que soporta millones de vectores y búsqueda aproximada eficiente.

Anexo — Variables de entorno relevantes

```
# LLM
OPENAI_API_KEY=sk-...
OPENAI_MODEL=openai:gpt-4o-mini
OPENAI_TEMPERATURE=0.2
OPENAI_MAX_TOKENS=1500

# Vector store
DATABASE_URL=postgresql://user:pass@host:5432/db # PGVector
PG_COLLECTION_NAME=carnicos_kb
OPENAI_EMBEDDING_MODEL=text-embedding-3-small

# HITL
HITL_ENABLED=false # true activa supervisión humana en todas las consultas RAG

# API
API_HOST=0.0.0.0
API_PORT=8000

# Trazabilidad (opcional)
LANGSMITH_TRACING=false
LANGSMITH_API_KEY=lsv2_pt_...
LANGSMITH_PROJECT=carnicos-kb-agent
```

*Informe generado sobre la rama **modulo-3-recomendacion-agentes-n8n**.*